

EXPERIMENT - 31

1. Problem Statement

Write a C program to perform the following menu-driven operations on a given 2D array:

- Find the sum of all elements in a matrix.
- Find the sum of each row of a matrix and store it in an array.
- Find the transpose of a matrix.
- Exit.

For a 2×3 matrix whose elements are 1 2 3 and 4 5 6, the sum of all elements is 21, the row sums are 6 and 15 with the row-sum array holding [6, 15], and the transpose is the 3×2 matrix 1 4 / 2 5 / 3 6.

2. Algorithm

Main Program

Step 1: START

Step 2: Read rows and cols, then read the rows $\times$ cols elements into the matrix a. This is the given 2D array and every option works on it.

Step 3: Display the menu once.

Step 4: Read choice.

Step 5: Using a switch statement on choice:

 If choice is 1, call sumOfAll().

 If choice is 2, call rowSums().

 If choice is 3, call transpose().

 If choice is 4, display "Exiting the program."

Step 6: Repeat from Step 4 while choice is not equal to 4.

Step 7: STOP

Function sumOfAll()

Step 1: Set sum = 0.

Step 2: Repeat for $i = 0$ to rows - 1:

 Repeat for $j = 0$ to cols - 1: add $a[i][j]$ to sum.

Step 3: Display sum.

Step 4: Return to the calling function.

Function rowSums()

Step 1: Repeat for $i = 0$ to rows - 1:

 Set sum = 0.

 Repeat for $j = 0$ to cols - 1: add $a[i][j]$ to sum.

 Set rowSum[i] = sum.

Step 2: Repeat for $i = 0$ to rows - 1: display "Sum of row $i + 1$ = rowSum[i]".

Step 3: Display the whole array rowSum inside square brackets, separated by commas.

Step 4: Return to the calling function.

Function transpose()

Step 1: Repeat for $j = 0$ to $\text{cols} - 1$:

Repeat for $i = 0$ to $\text{rows} - 1$: display $a[i][j]$.

Move to a new line.

Step 2: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Sum of all	2×3 matrix 1 2 3 / 4 5 6	Sum of all elements = 21	21	PASS
2	2. Row sums	the same matrix	Row 1 = 6, row 2 = 15, array [6, 15]	6 and 15, [6, 15]	PASS
3	3. Transpose	the same matrix	A 3×2 matrix 1 4 / 2 5 / 3 6	1 4 / 2 5 / 3 6	PASS
4	1. Sum of all	3×3 matrix 1 to 9	Sum of all elements = 45	45	PASS
5	2. Row sums	the same 3×3 matrix	[6, 15, 24]	[6, 15, 24]	PASS
6	3. Transpose	the same 3×3 matrix	1 4 7 / 2 5 8 / 3 6 9	1 4 7 / 2 5 8 / 3 6 9	PASS
7	2 and 3	1×4 matrix 5 10 15 20	Array [50], and the transpose is one column	[50], one column	PASS
8	4. Exit	choice = 4	Exiting the program, and the loop ends	Program ended	PASS

4. Source Code

matrixops.c

```
/* Experiment 31. Menu driven program on one given 2D array. The
   matrix is read once at the start, and every option in the menu works
   on that same matrix. */

#include <stdio.h>
#define MAX 20

void sumOfAll(int a[][MAX], int rows, int cols);
void rowSums(int a[][MAX], int rows, int cols);
void transpose(int a[][MAX], int rows, int cols);

int main(void)
{
    int a[MAX][MAX], rows, cols, i, j, choice;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("Enter the matrix elements:\n");
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &a[i][j]);

    printf("1. Find the sum of all elements in a matrix\n");
    printf("2. Find the sum of each row of a matrix and store it in an array\n");
    printf("3. Find the transpose of a matrix\n");
    printf("4. Exit\n");

    do {
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: sumOfAll(a, rows, cols); break;
            case 2: rowSums(a, rows, cols); break;
            case 3: transpose(a, rows, cols); break;
            case 4: printf("Exiting the program.\n"); break;
        }
    } while (choice != 4);

    return 0;
}

/* One running total for the whole matrix. Both loops together just
   walk over every element once. */
void sumOfAll(int a[][MAX], int rows, int cols)
{
    int i, j, sum = 0;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            sum = sum + a[i][j];

    printf("Sum of all elements = %d\n", sum);
}

/* sum has to go back to 0 at the start of every row, otherwise the
   rows keep adding on top of each other. The question asks for the
   totals to be stored, so they go into rowSum and are printed from
   there afterwards. */
void rowSums(int a[][MAX], int rows, int cols)
{
    int rowSum[MAX];
    int i, j, sum;

    for (i = 0; i < rows; i++) {
        sum = 0;
```

```

        for (j = 0; j < cols; j++)
            sum = sum + a[i][j];
        rowSum[i] = sum;
    }

    printf("Row sums:\n");
    for (i = 0; i < rows; i++)
        printf("Sum of row %d = %d\n", i + 1, rowSum[i]);

    printf("The row-sum array contains: [");
    for (i = 0; i < rows; i++) {
        printf("%d", rowSum[i]);
        if (i < rows - 1)
            printf(", ");
    }
    printf("]\n");
}

/* Here the outer loop runs over the columns and the inner one over the
rows, which is the opposite of normal printing, and that is what
turns a 2x3 matrix into a 3x2 one. At first I printed a tab after
every number and every line ended with an extra gap, so now the tab
goes before the number and is skipped for the first one. */
void transpose(int a[][MAX], int rows, int cols)
{
    int i, j;

    printf("Transpose of the matrix:\n");
    for (j = 0; j < cols; j++) {
        for (i = 0; i < rows; i++) {
            if (i > 0)
                printf("\t");
            printf("%d", a[i][j]);
        }
        printf("\n");
    }
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra matrixops.c -o matrixops` so that even the small warnings would show up, and it compiled clean in the first attempt itself. The first screenshot is the sample run given in the question.

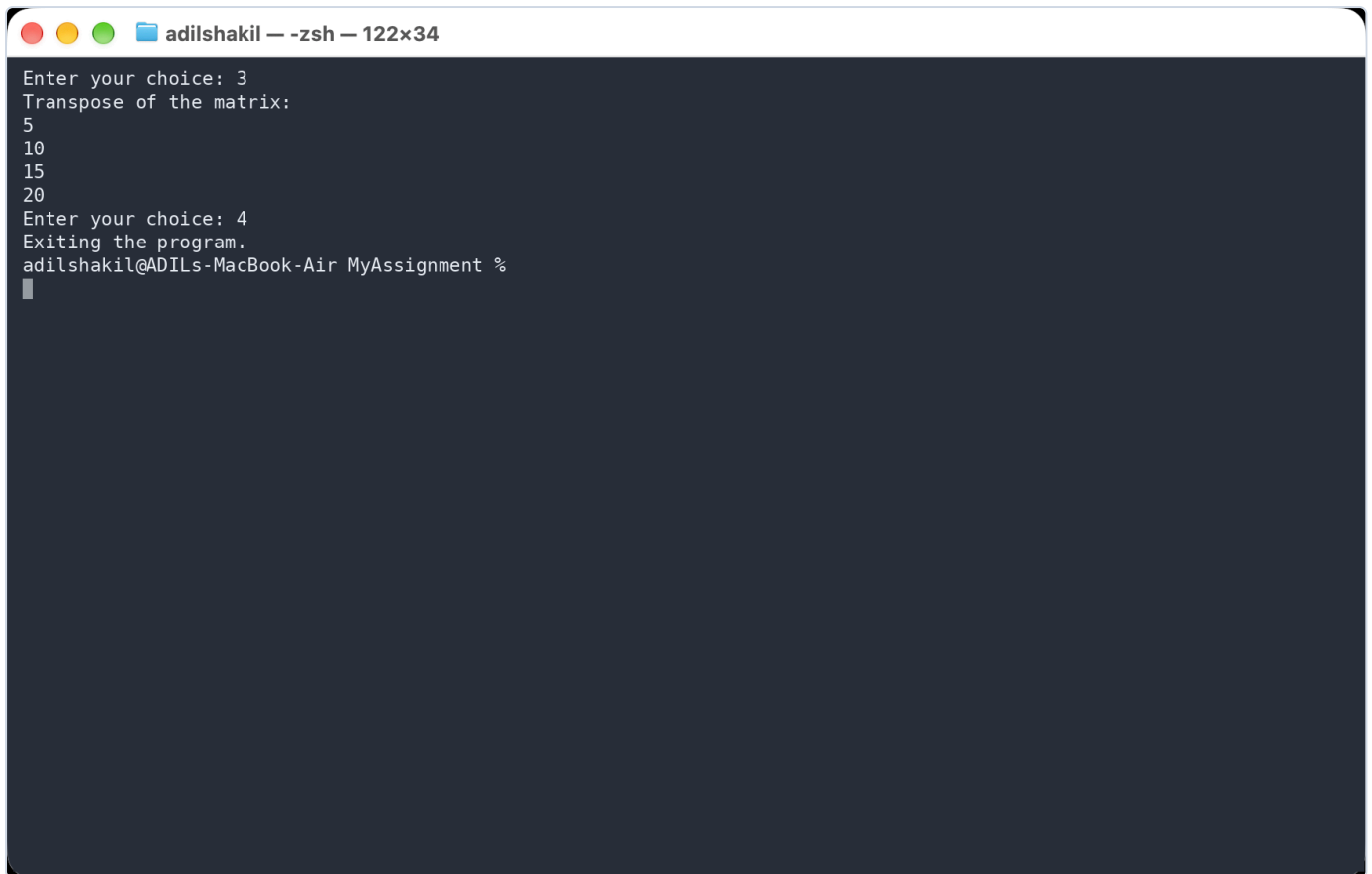
The Sample Run from the Question

```
adilshakil — zsh — 122x34
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra matrixops.c -o matrixops
adilshakil@ADILs-MacBook-Air MyAssignment % ./matrixops
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6
1. Find the sum of all elements in a matrix
2. Find the sum of each row of a matrix and store it in an array
3. Find the transpose of a matrix
4. Exit
Enter your choice: 1
Sum of all elements = 21
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]
Enter your choice: 3
Transpose of the matrix:
1      4
2      5
3      6
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

A Square Matrix and a Single-Row Matrix

```
adilshakil — zsh — 122x34
1 2 3
4 5 6
7 8 9
1. Find the sum of all elements in a matrix
2. Find the sum of each row of a matrix and store it in an array
3. Find the transpose of a matrix
4. Exit
Enter your choice: 1
Sum of all elements = 45
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
Sum of row 3 = 24
The row-sum array contains: [6, 15, 24]
Enter your choice: 3
Transpose of the matrix:
1      4      7
2      5      8
3      6      9
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment % ./matrixops
Enter the number of rows and columns: 1 4
Enter the matrix elements:
5 10 15 20
1. Find the sum of all elements in a matrix
2. Find the sum of each row of a matrix and store it in an array
3. Find the transpose of a matrix
4. Exit
Enter your choice: 2
Row sums:
Sum of row 1 = 50
The row-sum array contains: [50]
```

Transpose of the Single-Row Matrix, and Exit

A terminal window titled 'adilshakil — zsh — 122x34' with a dark background. The text inside shows a program running. It prompts 'Enter your choice: 3', then 'Transpose of the matrix:', followed by the numbers 5, 10, 15, and 20 on separate lines. Then it prompts 'Enter your choice: 4', followed by 'Exiting the program.' and the shell prompt 'adilshakil@ADILs-MacBook-Air MyAssignment %' with a cursor.

```
adilshakil — zsh — 122x34
Enter your choice: 3
Transpose of the matrix:
5
10
15
20
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All eight test cases came out right and the first run matches the sample output in the question line for line. The transpose was the only part I had to stop and think about. The outer loop has to run over the columns and the inner one over the rows, which is the opposite of how a matrix is normally printed, and that by itself turns a 2×3 matrix into a 3×2 one, so a second matrix is not needed at all. I did slip up there once: I printed a tab after every number, so every line finished with an extra gap. Printing the tab before the number instead, and skipping it for the first one, sorted it out.